

摘 要

随着校园外卖需求的快速增长，配送效率成为影响用户体验的关键因素。针对校园环境建筑密集、道路复杂、订单集中等特点，本文设计并实现了一套基于路径优化的校园外卖配送系统。系统以湖北大学校园为应用场景，以嘉慧园食堂为唯一配送中心，面向普通用户、骑手和管理员三类角色提供全流程服务。本系统基于路径优化技术，在这方面，本系统利用旅行商问题对多订单配送规划进行建模，采用遗传算法结合 2-opt 局部搜索进行最优化求解。其中，遗传算法采用锦标赛选择、顺序交叉和多种变异策略等算子。通过遗传算法实现全局搜索，2-opt 算法则通过消除控制路径交叉的方式进行局部精细优化。系统引入动态调度模块以应对配送途中新增订单的情形，模块采用部分冻结策略和最优插入算法，实现实时路径调整，并通过定期全局微调保证路线质量。本系统使用 Flask 框架构建，采用应用工厂模式和蓝图机制实现模块化开发。通过 OpenStreetMap 提供数据构建校园真实路网数据模型，通过 Dijkstra 算法计算兴趣点之间最短路径并构建距离矩阵。数据库采用 SQLAlchemy ORM 进行对象关系映射，以支持订单状态机管理并对骑手状态进行实时追踪。测试结果表明，系统能够有效降低配送总距离，提升配送效率。本系统对封闭式园区配送系统的设计具有一定参考价值。

关键词： 校园外卖配送；路径优化；遗传算法；动态调度；旅行商问题

ABSTRACT

With the rapid growth of campus food delivery demand, delivery efficiency has become a key factor affecting user experience. Considering the characteristics of campus environments such as dense buildings, complex roads, and concentrated orders, this paper designs and implements a campus food delivery system based on route optimization. The system uses Hubei University campus as the application scenario and Jiahuayuan Canteen as the sole delivery center, providing full-process services for three roles: ordinary users, couriers, and administrators. The system is based on route optimization technology. In this regard, it models multi-order delivery planning using the Traveling Salesman Problem and employs a combination of Genetic Algorithm and 2-opt local search for optimal solutions. Specifically, the genetic algorithm uses operators such as tournament selection, order crossover, and multiple mutation strategies. The genetic algorithm achieves global search, while the 2-opt algorithm performs local fine-tuning by removing path intersections. The system introduces a dynamic scheduling module to handle newly added orders during delivery. This module employs a partial freezing strategy and an optimal insertion algorithm to achieve real-time route adjustment, and ensures route quality through periodic global fine-tuning. The system is built using the Flask framework, employing the application factory pattern and blueprint mechanism for modular development. Campus real road network data is constructed based on data provided by OpenStreetMap, and the shortest paths between points of interest are calculated using Dijkstra's algorithm to build a distance matrix. The database uses SQLAlchemy ORM for object-relational mapping, supporting order state machine management and real-time tracking of courier status. Test results indicate that the system can effectively reduce total delivery distance and improve delivery efficiency. This system provides a certain reference value for the design of closed-campus delivery systems.

KEY WORDS: Campus Food Delivery; Path Optimization; Genetic Algorithm; Dynamic Scheduling; Traveling Salesman Problem

目 录

摘 要.....	I
ABSTRACT.....	II
目 录.....	III
1 引言.....	1
1.1 课题背景和意义.....	1
1.2 研究现状.....	1
1.3 本文研究内容.....	2
2 校园外卖配送系统需求分析.....	4
2.1 系统用例模型分析.....	4
2.1.1 参与者识别.....	4
2.1.2 各个角色用例分析.....	4
2.2 功能性需求分析.....	5
2.2.1 注册登录.....	5
2.2.2 普通用户端功能详细描述.....	5
2.2.3 骑手端功能详细描述.....	6
2.2.4 管理员端功能详细描述.....	6
2.3 非功能性需求分析.....	7
2.3.1 性能需求.....	7
2.3.2 可用性需求.....	7
2.3.3 安全性需求.....	7
3 基本设计原理.....	9
3.1 路网建模.....	9
3.1.1 路网的图模型表示.....	9
3.1.2 兴趣点的节点映射.....	9
3.1.3 最近节点映射.....	9
3.2 最短路径算法.....	10
3.3 旅行商问题.....	10
3.3.1 问题定义.....	10
3.3.2 问题建模.....	10
3.4 遗传算法.....	10
3.4.1 基本思想.....	10

3.4.2 染色体编码	11
3.4.3 适应度函数	11
3.4.4 选择算子——锦标赛选择	11
3.4.5 交叉算子——顺序交叉	12
3.4.6 变异算子	13
3.4.7 精英保留与早停机制	13
3.5 2-opt 局部搜索	13
3.6 动态调度	14
3.6.1 问题背景	14
3.6.2 部分冻结策略	14
3.6.3 最优插入过程	14
3.7 系统架构相关原理	14
3.7.1 Flask 工厂模式	15
3.7.2 蓝图与角色分离	15
3.7.3 订单状态机	15
4 系统设计	16
4.1 系统总体设计	16
4.1.1 设计目标	16
4.1.2 业务流程设计	16
4.1.2 系统架构设计	17
4.2 功能模块设计	17
4.2.1 用户模块	18
4.2.2 骑手模块	18
4.2.3 管理员模块	18
4.2.4 配送优化模块	18
4.3 数据库设计	19
4.3.1 用户表	19
4.3.2 订单表	19
4.3.3 批次表	19
4.3.4 骑手表	20
4.3.5 兴趣点表	20
4.4 地图模块设计	20

4.4.1 路网图构建	20
4.4.2 坐标吸附	21
4.5 路径优化模块设计	21
4.5.1 问题建模	21
4.5.2 遗传算法预处理（距离矩阵构建）	21
4.5.3 遗传算法设计	22
4.5.4 2-opt 局部搜索算法设计	24
4.6 动态调度模块设计	24
4.6.1 骑手状态模型	24
4.6.2 动态调度法	24
4.6.3 定期全局微调	25
5 系统实现	26
5.1 注册登录系统的实现	26
5.1.1 注册功能实现	26
5.1.2 用户登录功能实现	26
5.1.3 核心实现方法	27
5.2 用户端的实现	27
5.2.1 普通用户	27
5.2.2 骑手	29
5.2.3 管理员	30
5.2.4 核心实现方法	34
6 系统测试	35
6.1 注册登录模块测试	35
6.2 普通用户端功能测试	35
6.3 骑手端功能测试	36
6.4 管理员端功能测试	36
7 总结与展望	38
7.1 工作总结	38
7.2 工作展望	38
参考文献	40
致 谢	错误!未定义书签。

1 引言

1.1 课题背景和意义

校园外卖作为一种新型就餐方式，因学校统一化管理及下课时间集中，呈现出订单短时间内暴增的特点。如今，大学校园已经成为第三方平台外卖配送的主要区域。以美团外卖、饿了么为代表的 O2O 网上订餐平台首先抢占校园市场，受到高校学生的普遍欢迎。由于大学校园内建筑密集，出入口有限，故校园内各处线路错综复杂，而上下课高峰时人流极为拥挤，因此骑手一次配送过程很可能要送达若干外卖订单。由此自然地引出提高配送效率、保证食品质量、保障配送安全三者的内在需求：骑手宜主动、合理地解决配送线路问题，尽量减少单次配送里程及总耗时。若没有科学合理的路径优化策略，配送时间必然延长甚至出现配送延误，而冬季饭菜极易变凉，夏季冰饮甜点极易融化，二者都极大增加食品口感、风味受损的风险，也极易引起顾客投诉，最终损害商家声誉，直接导致骑手收入损失。由于本课题对所论问题及背景已有十分清楚的认识，因此本文自然、妥帖地设计了基于路径规划的校园外卖配送系统，又用遗传算法对配送路线做了优化，切实提高效率，降低成本及风险。

1.2 研究现状

由于高校外卖需求迅猛增长，又考虑到校园环境的种种特殊性，故如何建设安全、高效、智能的校园专属外卖系统已成为当前研究的热点，国内外学界对系统架构、配送路径优化、用户需求满足诸种问题已有充分讨论，尤以路径规划算法的研究最为突出。

在校园外卖系统的整体设计方面，李章恒（2022）^[1]在《校园外卖系统设计与实现》中对校园外卖系统的整体设计做了十分清晰、有层次的阐述：先提出了基于强化学习的个性化推荐算法，再据此设计移动端及 Web 端的校园外卖平台，更难得的是将食品安全监管、用户个性化需求两者很好地结合起来，由此自然、合理地构建起安全可控的封闭式订餐环境。因此其关于校园外卖系统整体架构的设计有极佳的参考价值。不过也毋庸讳言，现有工作集中于推荐策略及平台功能，没有对配送过程中的路径规划问题进行讨论。

配送路径的合理规划是提高外卖整体服务效率、保证服务品质的根本，因此围绕时间窗约束及多配送员协同调度问题，学界已有十分丰富、有层次的算法研究。赵家儒（2022）^[2]在《外卖配送车辆路径规划》中系统、严谨地提出了以时间窗为约束的路径优化模型，又自然、巧妙地设计了一种改进遗传算法，用染色体修复算子解决“先取后送”约束下产

生的不可行解问题，同时引入自适应策略增强算法稳定性。实验结果也十分清楚地表明，所提算法在单人、多人协同配送两种场景下都优于传统遗传算法及蚁群算法，故而也是校园外卖路径优化极好的算法支撑。

龚艺等(2019)^[3] 用遗传算法对外卖配送中车辆调度问题作了十分自然、合理的建模：先建立路径最短的车辆路径模型，再加入时间窗约束，最后以国内某外卖商家的实际数据为基础做了严谨的仿真实验，论证了算法的有效性。

在配送路径优化的算法层面，除了遗传算法以外，粒子群优化算法同样受到了广泛关注。Araújo 与 Barboza 在^[4]中对旅行商问题做了研究，把原来用来连续优化的粒子群算法正确地运用到到离散的组合优化的场景中：他们把粒子位置编码成城市的访问排列，通过引入 2-opt 和 3-opt 局部搜索，改进了粒子更新后的解的质量，并在标准旅行商问题测试实例上和传统遗传算法以及模拟退火算法进行了比较。实验结果表明，这一算法在中小规模问题上具有良好的求解效果，不过，在大规模实例中容易陷入局部最优。所以作者提出，未来可以通过与其他元启发式方法杂交来进一步提升性能。这一研究为本课题在校园外卖路径优化中综合运用遗传算法及其局部搜索策略提供了参考。

唐传茵等^[5]（2024）对结合改进蚁群算法（IACO）及大规模邻域搜索（LNS）的路径优化问题作了十分清楚、有层次的论述：先用 IACO 得到初始路线，再用 LNS 做精细调整，因而很自然地得到了其在不同订单量下都表现稳定、能减少配送超时、有利于平衡骑手负荷诸种结论。值得重视的是，作者也客观地指出目前尚无完整的落地应用平台。

本课题以已有结果为基础，把改进后的路径优化算法自然、合理地与系统平台开发结合起来，因而很自然地提出了三个明确、有层次的目标：第一，建立适合校园地形特点的配送路径模型，第二，开发具有订单管理、用户管理、路径优化诸种功能的 Web 端配套系统，第三，用仿真测试及实际场景测试来系统、严谨地验证算法及系统的性能。因此也自然地得到了高校校园外卖配送的切实可行的技术方案，更难得的是为其他封闭式园区场景的配送系统设计提供了可迁移的经验。

1.3 本文研究内容

随着校园生活的便利化，校园外卖成为学生日常的重要服务。为了提高配送效率，降低送餐时间，需要对配送员的送餐路径进行合理规划。

本课题拟设计一套校园外卖配送系统，系统包括：

- 1、校园地图建模模块（食堂、宿舍、配送中心及道路网络）
- 2、配送订单管理模块（支持多个订单同时处理）
- 3、配送路径规划模块（基于最短路径算法，结合配送顺序优化）
- 4、动态调度模块（可处理新增订单的路径调整）

目标是通过路径优化算法，实现高效、合理的校园配送路线规划，降低配送时间和距离，提高配送效率。

2 校园外卖配送系统需求分析

软件开发前，需要探讨校园外卖配送系统的需求分析，除了实现基本功能外，系统还应考虑提高用户体验和追求使用质量^[6]。

本系统以湖北大学校园为背景，构建一套面向校园场景的食堂外卖配送管理平台。系统以嘉慧园食堂为唯一配送起点，利用真实路网数据，结合遗传算法与局部搜索对配送路径进行优化，并支持动态调度以应对配送途中新增订单的场景。系统面向三类角色：普通用户、骑手、管理员。

2.1 系统用例模型分析

2.1.1 参与者识别

本系统分为三种用户：

下单外卖的在校师生用户，职责包括下单、查看订单状态、取消待处理订单；

负责配送的骑手用户，职责包括接收配送任务、取餐、按优化路线配送、确认送达；

管理员用户，负责管理地图兴趣点、触发路径优化、监控调度状态、管理用户账号等。

系统用户分配如图 2-1 所示。

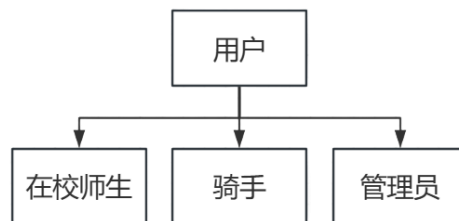


图 2-1 系统用户分配示意图

2.1.2 各个角色用例分析

下单外卖的普通用户（在校师生），需要登录系统的同时，还要能完成下单、查看订单状态、取消待处理订单的工作。

负责配送的骑手用户，职责包括接收配送任务、取餐、按优化路线配送、确认送达。

管理员用户，负责管理地图兴趣点、触发路径优化、监控调度状态、管理用户账号等。

本系统的三类角色用例如图 2-2 所示。

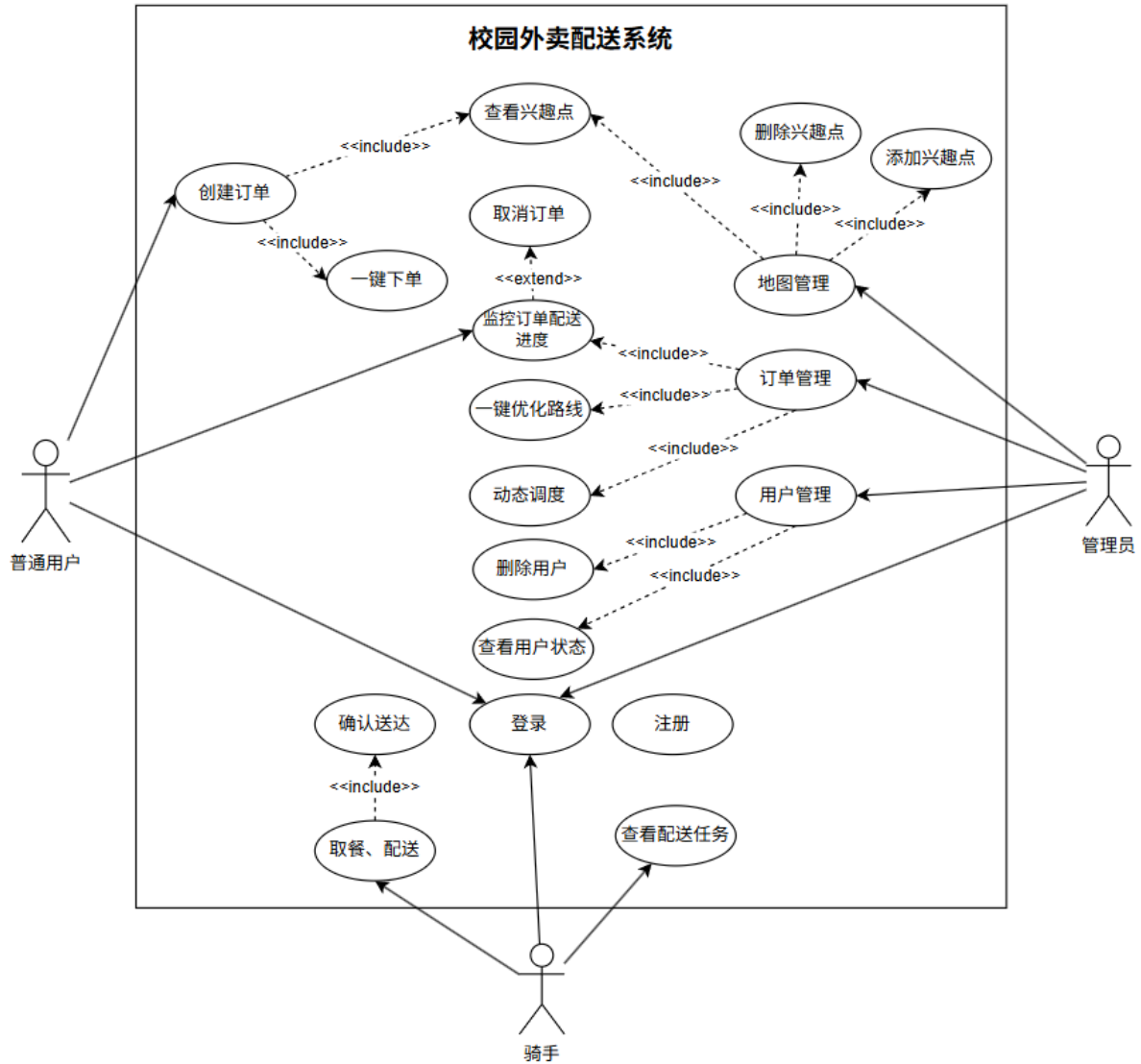


图 2-2 系统角色用例图

2.2 功能性需求分析

2.2.1 注册登录

注册登录模块用于注册成为系统用户，并实现登录功能。用户通过手机号进行注册，设置登录密码，选择身份角色（普通用户、骑手或管理员）完成账号创建。已注册用户输入手机号和密码即可登录系统。

2.2.2 普通用户端功能详细描述

普通用户所需要的功能主要包括创建订单、查看订单进度以及删除订单等，各个功能

的详细描述如下：

1、浏览配送地点：用户登录后进入系统主界面，可以查看校园内所有可配送的目的地列表。系统自动过滤食堂类地点，仅展示宿舍、教学楼、图书馆等非食堂类地点作为可选配送目的地。

2、创建订单：用户选择配送目的地后，系统自动将嘉慧园食堂设为配送起点，用户确认后即可提交订单。订单提交后进入待接单状态，等待系统分配骑手进行配送。

3、查看订单：用户可以查看个人历史订单记录，订单按创建时间倒序排列。每条订单显示当前配送状态、负责骑手姓名等信息。用户可实时追踪订单从待接单、已接单、已取餐、配送中到已送达的完整流程。

4、取消订单：在订单处于待接单状态时，用户可以选择取消订单。一旦订单被分配骑手进入已接单状态，则不可再取消。

2.1.3 骑手端功能详细描述

骑手端功能包括接受配送任务和执行配送任务等，各功能的详细描述如下：

1、查看配送任务：骑手登录后可以查看系统分配给自己的所有配送订单。订单按照系统优化的配送顺序排列，当前正在配送的订单会有明显标记，方便骑手识别。

2、一键取餐：骑手到达食堂后，可以批量确认取餐操作，将分配给自己的所有已接单订单统一标记为已取餐状态。

3、出发配送：骑手完成取餐后，一键确认出发配送，系统将所有已取餐订单更新为配送中状态，并开始记录配送进度。

4、确认送达：骑手到达每个配送地点后，逐单确认送达。每确认一单，该订单状态即更新为已送达。仅当前正在配送的订单显示确认送达按钮，防止误操作。

2.1.4 管理员端功能详细描述

管理员负责对地图、用户和配送路线分配等模块的管理，其各个功能的详细描述如下：

1、地图管理：管理员可以在地图上查看校园路网全貌，所有兴趣点的位置标记，当前所有订单的分布位置，以及各骑手的实时配送路线。

2、地点管理：管理员可以在地图上点击选点，添加新的配送目的地，或对已有地点进行修改和删除操作。系统确保食堂作为唯一配送起点的约束条件。

3、优化参数配置：管理员可以设置路径优化的相关参数，包括骑手数量、种群大小、迭代次数、变异率、交叉率，以及是否启用局部搜索优化。

4、路径优化：管理员触发路径优化后，系统对当前待处理订单进行全局路径规划，计算最优配送方案。优化完成后展示各骑手的配送路线和优化效果。

5、调度监控：管理员可以实时监控所有骑手的配送进度，查看每个骑手已完成配送的节点数量和总节点数量，掌握整体配送效率。

6、用户管理：管理员可以查看系统中所有注册用户的信息，对普通用户和骑手账号进行启用或禁用操作。管理员账号受系统保护，不可被禁用。

2.3 非功能性需求分析

2.3.1 性能需求

系统界面加载、地图数据加载在常规网络下都要做到流畅、无卡顿，因此系统要在用餐高峰时能自然、合理地支持预设数量的并发用户操作，界面响应要稳定可靠。路径优化计算必须在规定的时间阈值内完成，否则就会延误订单分配，影响骑手接单时效性。因此订单状态、骑手实时位置等信息应该按合理周期自动刷新，做到各相关方所见信息真正同步。

2.3.2 可用性需求

系统界面设计应该简洁明快，各功能安排都要合乎用户一般操作习惯。骑手端界面应能很自然地适应移动设备触控场景，所用按钮的尺寸应该有利于单手操作，取餐、出发、送达诸种流程都以分步引导式交互方式呈现，从而尽可能地降低误操作的概率。系统对用户的错误操作也要给出清楚、妥帖的提示，避免误触造成不良后果。同时系统应当兼容主流浏览器。

2.3.3 安全性需求

系统要有完善的身份认证机制。用户登录过程中，系统先校验用户登录凭证，只让注册用户进入系统，而普通用户、骑手、管理员各角色只能访问自己有权限的功能模块，以便系统能自然、合理地拦截越权请求。更重要的是，用户密码等敏感信息都要加以加密存储，杜绝泄露^[7]。订单取消、账号禁用等敏感操作都需二次确认。管理员账号作为系统最高权

限账号，理应受到最严格、最充分的保护。

3 基本设计原理

3.1 路网建模

3.1.1 路网的图模型表示

本系统对校园道路网进行抽象，得到一个无向加权图 $G = (V, E, W)$ [8]。其中 V 为表示节点的集合，每个元素代表路网交叉口或关键位置节点； E 为表示边的集合，每个元素代表两节点之间的双向道路； $W: E \rightarrow R^+$ 为表示各边权重的函数，边的权重为该路段的实际长度（单位：米）。

3.1.2 兴趣点的节点映射

由于校园内所有配送服务有关的地点（食堂、教学楼、宿舍楼等）都应合理地当作兴趣点来处理，故每个兴趣点都用 `node_index` 字段与路网图 G 中的节点自然绑定。为了简化系统设计，把注意力放在路径规划算法的设计上，系统把嘉慧园食堂定为全部配送任务的唯一起点，节点编号设为 `CANTEEN_NODE_ID = 19`。

3.1.3 最近节点映射

为了便于用户的使用，系统给出了最近节点映射的功能。

路网中的各个节点都带有明确的地理坐标（经度 x ，纬度 y ）。因此用户在地图上点击任意位置时，系统可以用 Haversine 公式直接计算这个点与所有路网节点之间的球面距离，再把用户点击坐标“吸附”到最近的合法路网节点，由此完成用户端的节点选择，同时确保了每个配送目的地的可达性。

下面写出 Haversine 公式如下：

$$d = 2R \cdot \arctan 2(\sqrt{a}, \sqrt{1-a}) \quad (3.1)$$

$$a = \sin^2 \frac{\Delta \phi}{2} + \cos \phi_1 \cos \phi_2 \sin^2 \frac{\Delta \lambda}{2} \quad (3.2)$$

其中 d 表示两点之间球面距离， $R = 6371000$ 米为地球半径， ϕ_1 、 ϕ_2 为两节点的纬度， $\Delta \phi$ 表示两节点的纬度之差， $\Delta \lambda$ 表示两节点之间经度差， a 为中间变量。

3.2 最短路径算法

Dijkstra 算法是求解单源最短路径的经典算法，它基于贪心策略，适用于所有边权非负的有向图和无向图^[9]。其核心思想是：对于图 $G = (V, E, W)$ 先维护一个到源点最短距离已确定的节点集合 S ，而后从尚未确定最短距离的节点集合 $S' = E/S$ 中，选取距离源点最近的节点加入 S ，并利用此节点更新其邻居节点到源点的最短距离，不断重复前述过程，直至所有节点均被处理。算法的时间复杂度为 $O((|V| + |E|) \log V)$ （使用优先队列实现）。

本系统利用此算法构建距离矩阵，作为后续遗传算法的输入。

3.3 旅行商问题

3.3.1 问题定义

多订单配送路径规划本质上是个旅行商问题，即给定 n 个配送地点和一个出发点^[10]，求一条经过所有地点恰好一次、总距离最短的回路。

旅行商问题是一个经典的 NP-hard 问题，当配送地点 n 的数量较大时，其解空间为 $n!$ 种排列，因此精确求解方法是不可行的。本系统采用启发式算法——2-opt 优化的遗传算法——在可接受的时间内求得高质量的近似最优解。

3.3.2 问题建模

在本系统中，旅行商问题被建模为：节点 0 表示食堂（旅行起点与终点，全程固定）；节点 1~ $n-1$ 表示各个配送目的地；目标函数定义为最小化路线总距离（其中 σ 为访问顺序的排列）：

$$\sum_{i=0}^{n-1} d_{\sigma(i), \sigma((i+1) \bmod n)} \quad (3.3)$$

其中 n 表示路线上节点数量。 $\sigma(i)$ 表示本路径中第 i 个被访问的节点的编号。 $d_{i,j}$ 表示距离矩阵中的元素 $d[i][j]$ ，即本章 3.2 节所计算的 i 点到 j 点的最短距离，这里 i 和 j 是节点编号。这里 $(i + 1) \bmod n$ 表示取模运算，确保食堂作为节点的起点和终点会在求和开始和结束时被访问到并加入计算。

3.4 遗传算法

3.4.1 基本思想

遗传算法是一种模拟自然界生物进化过程的元启发式搜索算法，由 Holland 于 1975 年提出^[11]。其核心思想是：通过对候选解种群的选择、交叉和变异操作，模拟“适者生存”的自然选择机制，使种群一代代不断演化收敛，直至得到最优或近似最优解。

遗传算法的主要特点是可以直接对种群空间中的多个解进行搜索，对函数的求导及其连续性的判定没有约束，具有良好的鲁棒性和更全面的全局搜索寻优能力^[12]。

3.4.2 染色体编码

每条染色体为本次配送路线，编码为整数排列，即目标节点序列。例如染色体编码[0, 1, 2, 3, 4]，其中 0 代表食堂，固定排列在首位，其余数字序列代表各个配送目标节点的访问顺序。这种编码方式能够天然保证每个目标节点在一条染色体中恰好出现一次，避免无效解的产生。

3.4.3 适应度函数

适应度定义为路线总距离的倒数：

$$f(\text{route}) = \frac{1}{\text{TotalDistance}(\text{route})} \quad (3.4)$$

其中 $f(\text{route})$ 为适应度函数，其输入为路线顺序，输出为该路线的适应度。 $\text{TotalDistance}(\text{route})$ 为总距离计算函数，其输入为路线顺序，输出为该路线总距离。所以路线越短，适应度越高，被选中并参与繁殖的可能性就越大。若路线包含无效节点（距离为 INF），则适应度为 0。

3.4.4 选择算子——锦标赛选择

本系统采用锦标赛选择（Tournament Selection），即每次从种群中随机抽取 k 个个体，选择其中适应度最高的个体作为父代。重复该过程两次，得到两个父代个体用于后续交叉操作。锦标赛选择的压力通过 k 值调控， k 值越大则高适应度个体胜出的概率越高，容易收敛但不利于种群多样性的保留， k 值越小则反之。本系统默认 k 值为 3，以图在收敛效率与多样性维持之间寻求平衡值^[13]。锦标赛选择过程如图 3-1 所示。

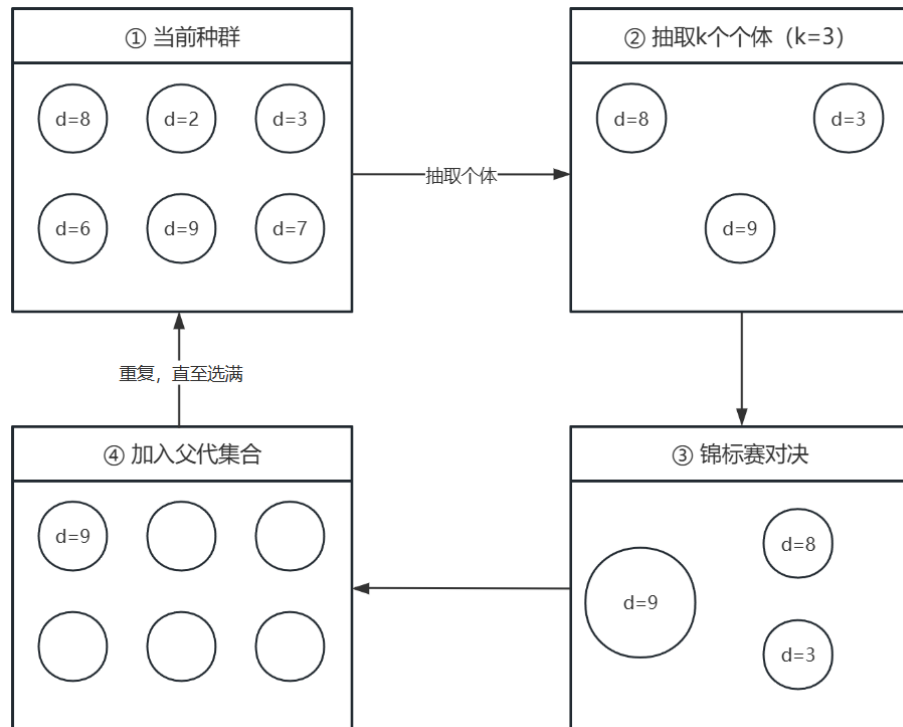


图 3-1 锦标赛选择示意图

3.4.5 交叉算子——顺序交叉

针对旅行商问题的排列编码特性，若直接采用单点或多点交叉，极易出现重复访问或遗漏节点。为此，本系统选用顺序交叉，其步骤如下：

- 1、从父代 P_1 中随机选取一段连续基因子串（不含食堂位置），将其复制到子代 C 的对应位置；
- 2、遍历父代 P_2 ，按原有顺序提取尚未出现在子代 C 中的基因；
- 3、将提取出的基因按顺序填入子代 C 中的剩余空位。

该算子继承了父代 P_1 的部分邻接关系，又保持了父代 P_2 的全局相对顺序，能够有效传递父代优势。食堂节点不参与交叉操作，以保证每条路线均以食堂为起终点。顺序交叉过程如图 3-2 所示。

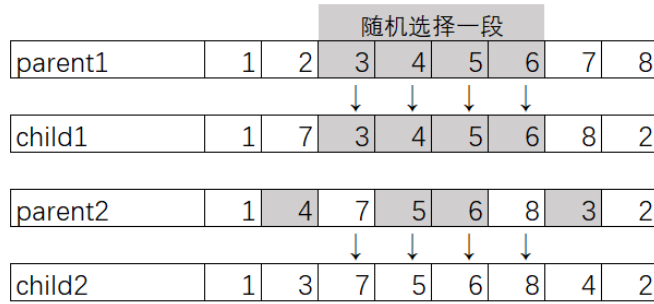


图 3-2 顺序交叉操作示意图

3.4.6 变异算子

变异算子以较小概率（本系统默认为 0.2）作用于交叉产生的子代个体，旨在引入新的基因结构，防止种群过早收敛于局部最优。本系统每次触发变异时，会从以下三种方式中随机选择一种执行^[14]：

- 1、交换变异：随机选择两个非食堂位置，交换这两个位置上的配送点编号；
- 2、插入变异：随机选择一个配送点，将其从当前位置移除，并插入到另一个随机选取的位置之后（或之前）；
- 3、反转变异：从染色体中随机选择两个端点，然后把两个端点之间的连续片段，也就是子路径中的所有节点的访问顺序整体翻转。

三种变异算子从不同程度上扰动了染色体结构：交换变异侧重于局部交换，插入变异改变单个节点的前后继关系，反转变异则能有效地改变较大范围内的片段的顺序（尤其适用于消除路径中的“交叉”现象）。三者的组合使用有效增强了种群在解空间中的探索能力。

3.4.7 精英保留与早停机制

每代进化时，当代最优个体直接保留到下一代（精英策略），以防最优解丢失。若连续 100 代无改进，则算法提前终止，以节省计算资源。

3.5 2-opt 局部搜索

2-opt 是针对旅行商问题的经典局部搜索算法。其核心思想是：在当前路线中选取两条不相邻的边 $(i, i + 1)$ 和 $(j, j + 1)$ ，将 $i + 1$ 到 j 之间的路段整体反转。若新路线总距离更短，则接受该次改变，在同一次调用中反复迭代直至无法继续用 2-opt 方法改进。2-opt 局部优化过程如图 3-3 所示。

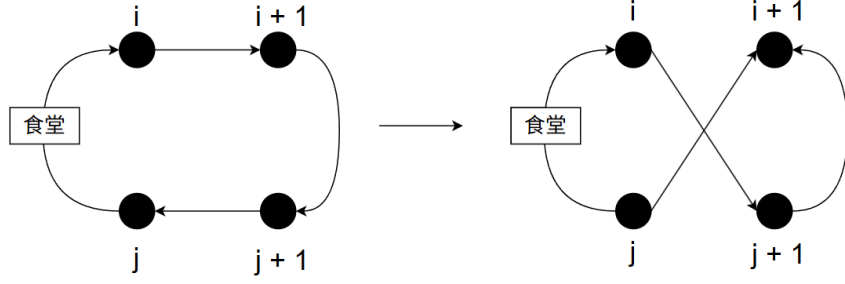


图 3-3 2-opt 局部优化示意图

2-opt 能有效消除路线中的"交叉"现象，是对遗传算法全局搜索的精细补充^[15]。

3.6 动态调度

3.6.1 问题背景

骑手在配送途中，经常会出现新增订单，如果只是在为骑手分配订单之前按批次静态分配，则配送途中的订单将不能处理，用户等待的时间就会增加。本系统引入动态调度器，能够在骑手在已经出发以后实时插入新的订单，从而提高配送效率，进而提升用户体验。

3.6.2 部分冻结策略

系统为每个骑手设置一个冻结指针，指针指向骑手正在执行配送的配送点。冻结指针之前的路段在后续的动态调度中不能再更改，但它之后的路段是可调整区间，允许动态插入新订单并重新进行优化。

3.6.3 最优插入过程

当新订单到达时，系统遍历所有骑手的可调整路线段，将新订单尝试插入每个可能的位置，计算插入后路线总距离增量的计算过程如公式 3.5 所示。

$$\Delta d = d_{prev,new} + d_{new,next} - d_{prev,next} \quad (3.5)$$

其中 Δd 表示路线总距离的增量。 $d_{i,j}$ 表示距离矩阵中的元素 $d[i][j]$ ，即本章 3.2 节所计算的 i 点到 j 点的最短距离，这里 i 和 j 是节点编号。 new 为新插入节点的待选位置，位于冻结指针之后， $prev$ 为其前一节点， $next$ 为其下一节点。选择使 Δd 最小的骑手和插入位置执行插入，时间复杂度为 $O(R \times N)$ （ R 为骑手数， N 为可调整节点数），可在毫秒级完成。

3.7 系统架构相关原理

3.7.1 Flask 工厂模式

本系统后端基于 Flask 框架构建，采用应用工厂模式，通过 `create_app()` 函数动态创建 Flask 实例，便于在不同环境（开发/生产）下使用不同配置，并避免循环导入的问题。

3.7.2 蓝图与角色分离

本系统设置三类用户角色：普通用户、骑手、管理员。其路由逻辑分别封装为独立的 Flask 蓝图，实现关注点分离。Flask 蓝图基本构成如表 2.1 所示。

表 2.1 Flask 蓝图

蓝图	URL 前缀	职责
user	/user	下单、查看订单状态
rider	/rider	接单、更新配送状态
admin	/admin	触发批次优化、监控调度器
delivery	/delivery	核心配送 API

3.7.3 订单状态机

订单生命周期通过有限状态机管理，状态转移关系定义于 `STATUS_TRANSITIONS` 字典中，防止非法状态跳转。

4 系统设计

4.1 系统总体设计

4.1.1 设计目标

本系统以湖北大学校园为应用场景，以“嘉慧园食堂”为唯一配送中心，面向校园内学生、骑手和管理员三类用户。实现从下单、批次优化、路线和订单分配到实时配送的全流程闭环服务。系统的核心设计目标如下：

- 1、路径最优化：利用遗传算法结合 2-opt 局部搜索，对多订单多骑手场景下的配送路线进行全局优化，最小化总行驶距离；
- 2、动态实时调度：支持配送进行中插入新订单；
- 3、角色权限隔离：系统严格区分普通用户、骑手和管理员三种角色，各个角色只能访问他们的权限范围以内的功能。
- 4、地理空间感知：基于 OpenStreetMap 真实路网数据构建校园地图，距离计算基于 Dijkstra 算法，而不是直线距离。

4.1.2 业务流程设计

从用户下单到路径优化再到骑手配送、送达和收货的整个业务流程需要管理员、骑手、平台（系统）和普通用户等多个角色的参与，下面是详细的业务流程说明。

- 1、用户下单流程：用户通过前端界面选择配送地点，然后提交订单，系统创建订单记录。
- 2、管理员优化流程：管理员查看待处理订单，启动批次优化。之后系统使用遗传算法计算最优路线并分配给骑手。
- 3、骑手配送流程：骑手接收到系统分发的配送任务后，到食堂取餐并批量更新订单状态，而后按优化路线逐个送达。

总体业务流程如图 4-1 所示。

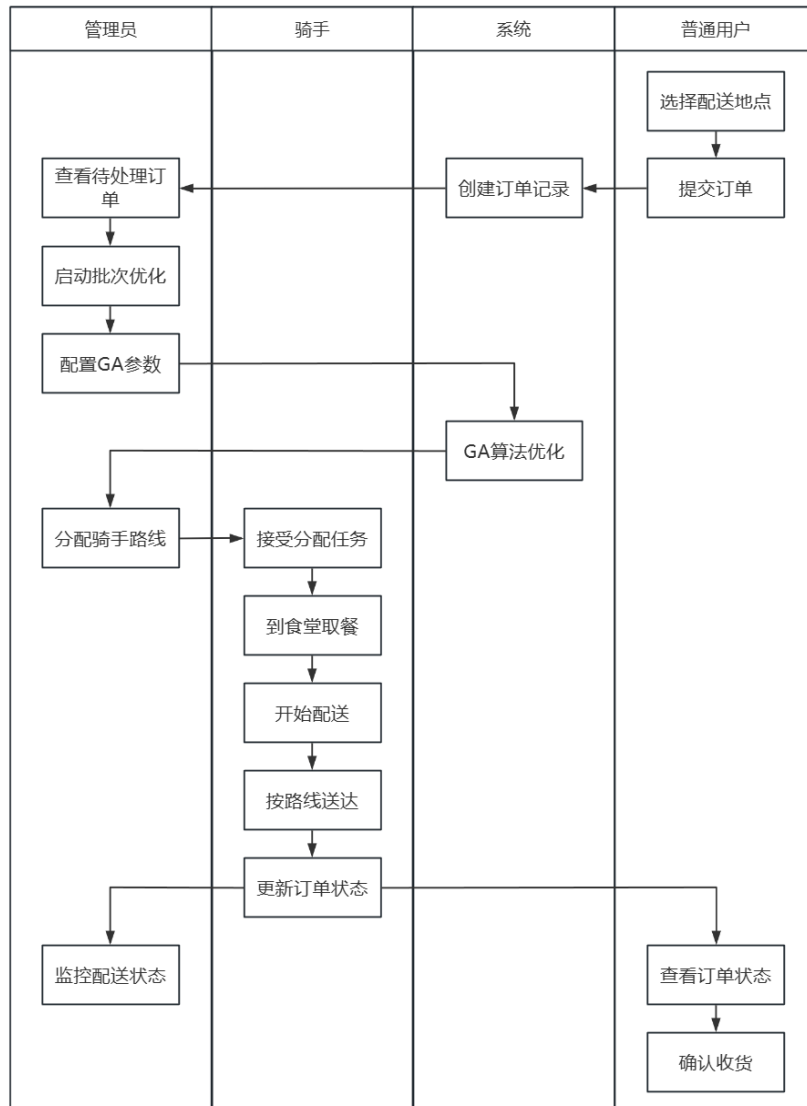


图 4-1 总体业务流程示意图

4.1.2 系统架构设计

系统架构采用 Flask 应用工厂模式，整体分为三层架构：

- 1、表示层：由 5 个 Flask 蓝图组成，负责 HTTP 请求路由与页面渲染，各蓝图之间相互独立，通过 `create_app()` 工厂函数统一注册。
- 2、业务逻辑层：封装核心算法和业务流程，包括订单生命周期管理、地图建模和计算逻辑、遗传算法优化和动态调度模块。
- 3、数据持久层：使用 SQLAlchemy ORM 操作 SQLite 数据库，使用 Flask-Migrate 管理数据库迁移。

4.2 功能模块设计

4.2.1 用户模块

普通用户（学生、教职工等）可执行以下操作：

- 1、下单：从预设兴趣点列表中选择配送目的地，提交订单；
- 2、订单追踪：实时查看订单当前状态。
- 3、订单状态包含“待接单”、“已接单”、“已取餐”、“配送中”、“已送达”。

4.2.2 骑手模块

骑手用户可执行以下操作：

- 1、查看分配路线：查看系统为其分配的当前批次配送路线以及各订单目的地；
- 2、更新订单状态：按配送进度逐步推进订单状态，每次状态更新同步出发调度器的冻结指针前进。

4.2.3 管理员模块

管理员拥有最高权限，可执行以下操作：

- 1、兴趣点管理：新增、编辑、停用校园配送地点（宿舍楼、学院楼、图书馆等）；
- 2、批次优化：出发遗传算法对当前所有待处理订单进行路线优化，生成最优配送批次；
- 3、调度器控制：启动、定制动态调度后台线程，监控实时调度状态；
- 4、用户管理：管理系统内所有账号。

4.2.4 配送优化模块

该模块是系统核心 API 层，对外暴露关键接口如表 4.1 所示。

表 4.1 配送优化模块接口表

接口路径	方法	功能描述
/delivery/optimize	POST	触发遗传算法批次优化
/delivery/scheduler/start	POST	启动动态调度器
/delivery/scheduler/insert_order	POST	动态插入新订单
/delivery/scheduler/state	GET	查询调度器实时状态

4.3 数据库设计

数据库的实体-关系图（E-R 图）如图 4-2 所示。

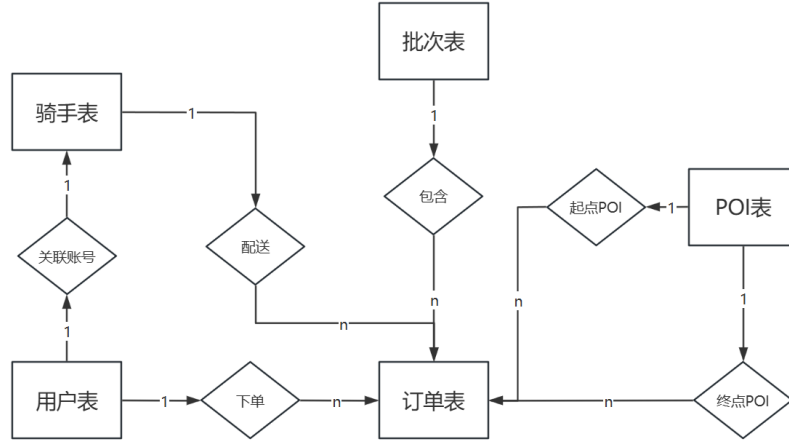


图 4-2 实体-关系图

4.3.1 用户表

采用单表多角色设计，通过 `role` 字段区分用户、骑手和管理员三种角色，降低数据库复杂性。密码使用 `werkzeug` 的 PBKDF2-SHA256 算法哈希存储，永不保存明文。

4.3.2 订单表

订单表是系统最核心的数据实体，关键字段设计如下：

- 1、`to_node_index`：目的地在路网图中的节点编号，作为路径计算的直接输入；
- 2、`status`：订单当前状态，受状态机约束；
- 3、`is_frozen`：标记该订单在动态调度中是否已被冻结（即是否已有骑手在配送或已完成配送）；
- 4、`insert_batch_id`：记录该订单是否通过动态插入方式加入已有批次。

4.3.3 批次表

每次遗传算法优化产生一个批次记录，存储优化结果的元数据：

- 1、`optimal_route_json`：序列化的最优路线节点序列；
- 2、`ga_params_json`：本次优化使用的算法参数；

3、total_distance: 优化后的最优距离（米）。

4.3.4 骑手表

骑手表通过 user_id 与 users 表一对一关联，扩展骑手专属属性（如接单状态等）。

4.3.5 兴趣点表

存储所有校园配送地点：

- 1、node_index: 路网节点编号；
- 2、lat: 节点纬度；
- 3、lon: 节点经度；
- 4、poi_type: 地点类型（食堂、教学楼、图书馆等）。

4.4 地图模块设计

4.4.1 路网图构建

本系统通过 OSMnx 库调用 OpenStreetMap 开放地图数据，以湖北大学为中心（经度 114.32819，纬度 30.57978），以 400 米为半径，获取骑手可通过的路网，以构建 NetworkX 的 MultiDiGraph 对象。

路网数据采用单例缓存模式进行管理，即路网图在首次请求时下载，之后该对象供所有模块共同使用，以避免重复的网络请求开销。

路网图构建效果如图 4-3 所示。



图 4-3 路网建模效果图

4.4.2 坐标吸附

当用户通过地图点击新增兴趣点时，系统使用 Haversine 公式计算点击坐标与路网中所有节点的球面距离，将其"吸附"到最近的合法路网节点，确保每个配送目的地在图中均可达。

4.5 路径优化模块设计

4.5.1 问题建模

本系统将多订单配送路线规划问题建模为旅行商问题。节点包括固定索引为 0 的食堂和各订单目的地，边权重为路网最短距离，目标是从食堂出发、经过所有配送节点后返回食堂的最短闭合路径，且食堂固定为路线起点。

4.5.2 遗传算法预处理（距离矩阵构建）

系统构建距离矩阵，对所有配送节点（食堂节点和订单目的地节点）两两利用 Dijkstra

算法，得到两点间实际骑行距离（米），并存入距离矩阵中对应位置。

对于 n 个配送节点 ($n \ll |E|$)，最终生成一个 $n \times n$ 的距离矩阵 D ，其中 $D[i][j]$ 表示节点 i 到节点 j 的最短路径长度。若两节点之间不存在路径可到达，则对应距离设置为 INF。

遗传算法优化器的输入是一个 $N \times N$ 的距离矩阵，其中 N 为本批次中配送地点数量（含食堂），矩阵中每个元素由 `GraphService.compute_distance_matrix()` 方法调用 NetworkX 中的 `nx.shortest_path(graph, source, target, weight='length')` 函数计算得出，代表节点 i 到节点 j 的实际路网最短距离（米）。

4.5.3 遗传算法设计

遗传算法设计中，每条染色体表示一种配送顺序排列，其表示为[食堂节点编号，订单 1 节点编号，订单 2 节点编号，...，食堂节点编号]。种群大小默认为 200。适应度取路线总距离的倒数，距离越短适应度越高，无效路线适应度直接置为 0。选择采用锦标赛方法，以适应路线规划这类适应度差异较大的问题。交叉使用擅长保留路线顺序优势的顺序交叉方法。变异以一定的概率随机选择交换、插入或反转三种操作之一^[16]。为了尽可能消除最优个体内部的路线交叉，每 10 代对当前最优路线执行 2-opt 局部搜索，消除路线交叉，提升优化的精度。若连续 100 代最优距离无改善则提前终止，以提高算法执行效率。

创建 `GeneticAlgorithmTSPWith2Opt` 类封装完整的遗传算法求解过程，类内部维护 `best_distances` 列表，记录每代最优距离，供前端绘制收敛曲线（如图 4-4 所示）。通过表 4.2 所示参数构造新对象。

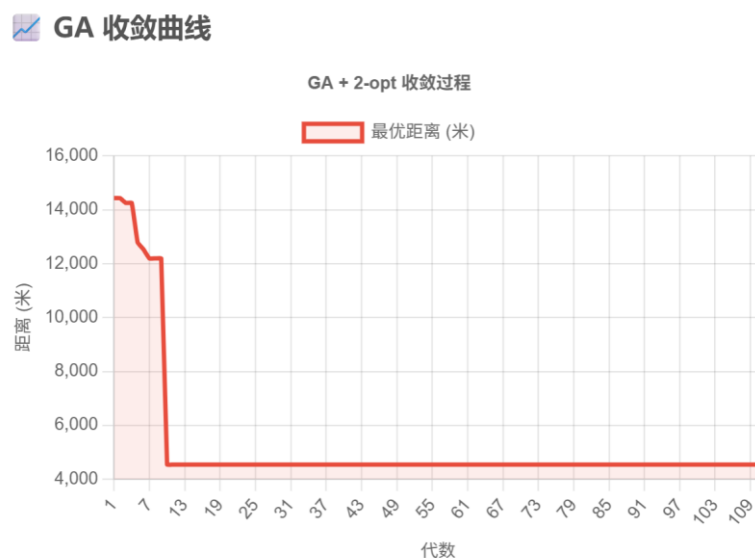


图 4-4 收敛曲线示例

表 4.2 构造函数参数

参数	默认值	说明
distance_matrix	无	$n \times n$ Numpy 数组
population_size	200	每代种群大小
generations	500	最大进化代数
mutation_rate	0.2	变异概率
crossover_rate	0.8	交叉概率
use_2opt	True	是否启用 2-opt
apply_2opt_interval	10	2-opt 触发间隔（代）

主循环 solve()每代执行流程如下：

- 1、计算全部个体的距离和适应度；
- 2、更新全局最优(best_route、best_distance)，若无改善则 no_improvement_count += 1；
- 3、每隔 apply_2opt_interval 代对 best_route 进行一次 2-opt 操作；
- 4、精英个体（best_route）副本直接放入新种群首位，然后通过选择、交叉、变异填充新一代种群。
- 5、重复步骤 1~4 的操作，直至重复总次数达到最大进化代数 generations，或 no_improvement_count>100 提前终止。

路线距离和适应度计算方法 calculate_distance(route)，将路线视为闭合回路，累加 distance_matrix[from][to]得到总距离，适应度定义为 $f_i = 1/d_i$ ，距离为 INF 或 0 的个体单独设置，以防止除零错误。

顺序交叉 crossover()的具体步骤：

- 1、以概率 crossover_rate 决定是否执行交叉；
- 2、从 range(1, size)中随机抽取两个位置 start, end；
- 3、子代 1 的[start:end]直接复制父代 1 的对应片段，其余位置按父代 2 基因出现的顺序填入。

变异操作以 mutation_rate 概率触发，随机选择三种方式之一：

- 1、交换变异：route[i], route[j] = route[j], route[i]，即仅交换两个位置的值；

2、插入变异：`city = route.pop(i); route.insert(j, city)`，即将某节点移动插入到新位置。

3、反转变异：`route[i: j] = reversed(route[i: j])`，利用 Python 切片赋值反转子段。

4.5.4 2-opt 局部搜索算法设计

系统每隔 `apply_2opt_interval` (默认 10) 代对当前的全局最优路线执行一次 2-opt 优化，这样的迭代最多连续进行 100 次。如果在 2-opt 的优化过程中找到了新的最短的路线，则立即更新全局最优路线并重置早停计数器。

2-opt 局部搜索是通过函数 `two_opt(route)` 实现的，系统利用了双层循环去遍历所有 (i, j) 元组（这里 $0 < i < j < n$ ），对每个 (i, j) 元组都做 `new_route[i:j] = reversed(new_route[i:j])` 的操作，所得到的新距离更短则接受，同时重置外层循环。最多迭代 100 轮。

2-opt 在遗传算法主循环中每隔 `apply_2opt_interval` 代触发一次，这里只对当前全局最优路线 `best_route` 调用 2-opt，不对种群的全体成员都做这样的处理。这样设计的理由是：对单个路线执行 2-opt 计算的时间复杂度为 $O(n^2)$ ，对整个种群计算则代价很高，且只对最优个体优化既能保证质量提升，有不会显著影响遗传算法运行速度。

遗传算法与 2-opt 具有相互促进的关系。遗传算法负责全局搜索，能在解空间中快速定位解的质量很高的一部分区域。2-opt 负责局部优化，能在遗传算法找到的解附近进一步消除冗余路径（以交叉迂回路径为主）。两者结合可以显著地提升解的质量。

4.6 动态调度模块设计

4.6.1 骑手状态模型

每个骑手的运行时状态由 `CourierState` 对象维护，核心属性如下：

`full_route`: 完整路线，格式为[食堂, 节点 1, 节点 2, ..., 食堂];

`order_db_ids`: 与 `full_route` 一一对应的订单数据库 id 列表;

`frozen_index`: 冻结指针，指向骑手已完成或正在执行的最后一个节点下标，冻结指针之前的路段不可修改。

4.6.2 动态调度法

按照部分冻结策略，使用 `get_adjustable_range()` 方法返回 `frozen_index + 1` 到倒数第二个节点（不含末尾食堂）之间的所有订单节点索引，即当前可重新排列的路段。

新订单到达时，使用 `insert_order()` 方法遍历所有骑手的可调整路段，选择最佳路段作为动态调度结果。

4.6.3 定期全局微调

后台线程每隔 60 秒（可配置）触发一次 `periodic_reoptimize()`，对每个骑手的可调整路段单独运行一次小规模遗传算法（种群 50，代数 30），进一步优化因动态插入导致的路线质量下降，该过程通过 `threading.Lock` 保证线程安全。

两层调度策略对比详情如表 4.3 所示。

表 4.3 配送优化模块接口表

层次	触发方式	算法	响应时间	优化范围
实时层	新订单到达	启发式算法	毫秒级	单次插入
定期层	后台定时	遗传算法和 2-opt	秒级	全局微调

5 系统实现

5.1 注册登录系统的实现

5.1.1 注册功能实现

师生和骑手进入注册页进行用户注册，输入用户名、手机号、密码和身份（普通用户/骑手）后进行注册，服务器收到表单内容等数据后调用用户注册方法，并返回注册成功/失败的结果。注册页面如图 5-1 所示。



图 5-1 用户注册页

5.1.2 用户登录功能实现

师生、骑手和管理员可输入手机号和密码进行登录，登录成功可自动跳转各自角色对应页。用户登录界面如图 5-2 所示。

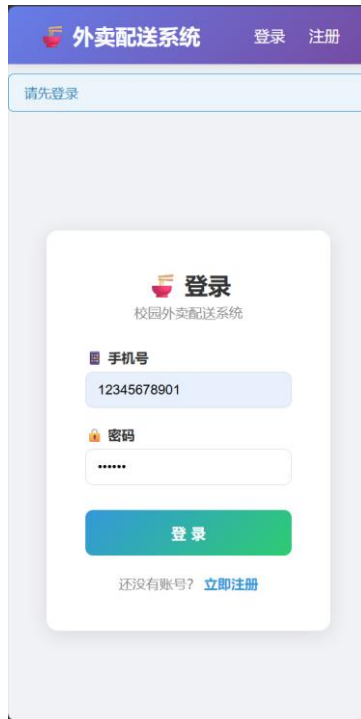


图 5-2 用户登录界面

5.1.3 核心实现方法

1、通过对象关系映射（ORM）的方法，将数据库 user table 映射为 Python 类 User，之后可通过操作 Python 来间接操作数据库；

2、调用 set_password()方法对密码进行单向哈希加盐，只保存哈希结果，实现密码安全存储；

3、利用 SQLAlchemy 的会话机制跟踪对象状态；

4、使用 login_user(user, remember=True)方法创建会话，并存入用户标识，会话由 Flask 签名保护。remember=True 会额外设置 cookie，实现登录状态保持。

5、根据用户角色和 next 参数返回不同的重定向。

5.2 用户端的实现

5.2.1 普通用户

普通用户页可针对兴趣点进行下单或查看历史订单。普通用户界面（工作台）如图 5-3 所示。



图 5-3 普通用户界面

通过下拉表单进行下单。对应界面如图 5-4 所示。



图 5-4 下单目的地选择菜单

对于已下单订单，可查看其配送状态。配送状态查看界面如图 5-5 所示。



图 5-5 查看配送状态

5.2.2 骑手

骑手在工作台可查看历史配送路线和送单统计信息。查看配送信息界面如图 5-6 所示。



图 5-6 查看配送信息

开始配送后，骑手工作台显示配送路线信息，送达后骑手可点击“确认送达”，而后配

送下一目标。查看配送路线界面如图 5-7 所示。



图 5-7 查看配送路线

5.2.3 管理员

管理员可点击地图添加新的兴趣点。点击地图或者手动输入经纬度，系统会根据地图上被点击位置的经纬度，通过第三章介绍的最近节点映射的方法自动“吸附”到最近合法路网节点。之后管理员就可以在工作台为其编辑名称、选择类型并添加描述。管理员也可以通过手动输入合法路网节点编号来确定新建兴趣点位置，并在输入基本信息后添加兴趣点。

新建兴趣点界面如图 5-8 所示，操作过程如图 5-9 所示。

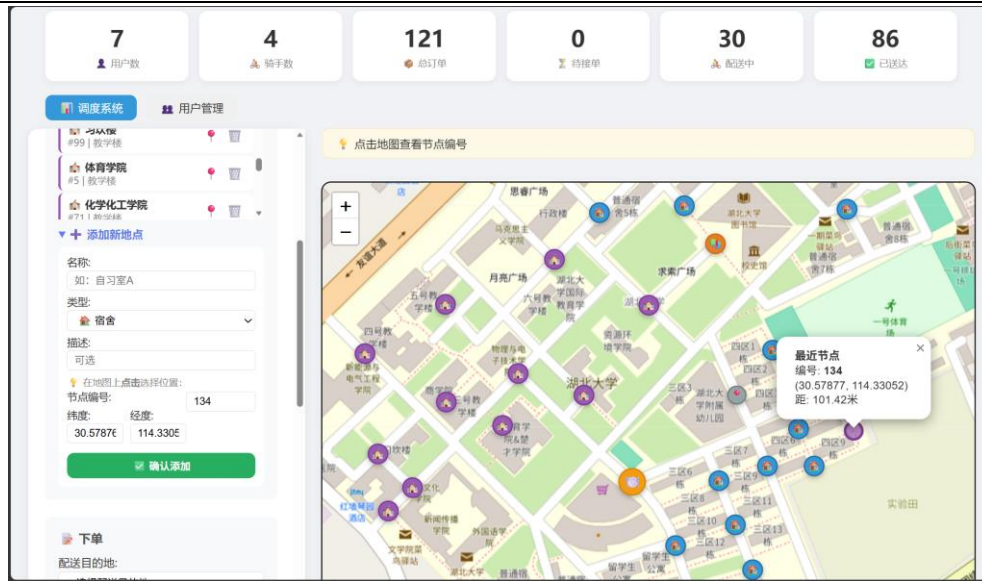


图 5-8 新建兴趣点界面



(a) 点击地图映射最近节点

(b) 编辑兴趣点信息

(c) 成功新建兴趣点

图 5-9 新建兴趣点过程

管理员点击优化 pending 订单后，经过少许优化时间，系统自动将优化后的线路分配给骑手，并在地图中显示路线。骑手端将收到路线并自动接单。优化 pending 界面样式如图 5-10 所示。地图中不同颜色的路线即对应骑手的配送路线。将骑手端界面显示的路线在地图中模拟绘制，如图 5-11 所示，展示了骑手 1 的分配路线情况，其中图 5-11 (a) 是骑手待处理订单顺序，如图所示，当前骑手正在执行 1 号订单（即 3 区 12 栋），接下来他要顺序处理编号为 2 的订单，这些订单的配送路径如图 5-11 (b) 所示，红色箭头代表其行进方向。不难看出，分配给骑手的配送顺序中少有路线交叉出现，说明 2-opt 优化在消除路径交叉上的有效性。

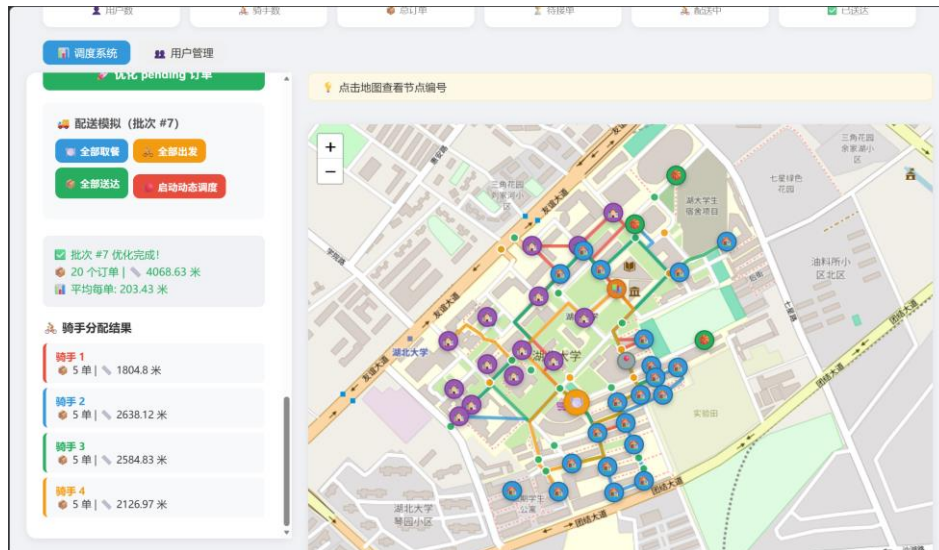
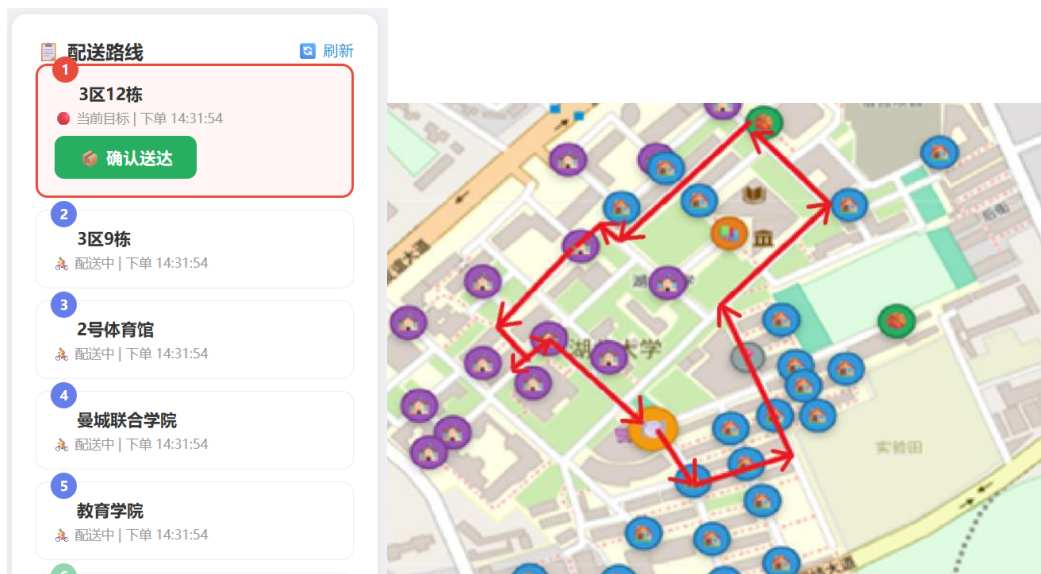


图 5-10 优化 pending 订单



(a) 骑手 1 配送路线界面

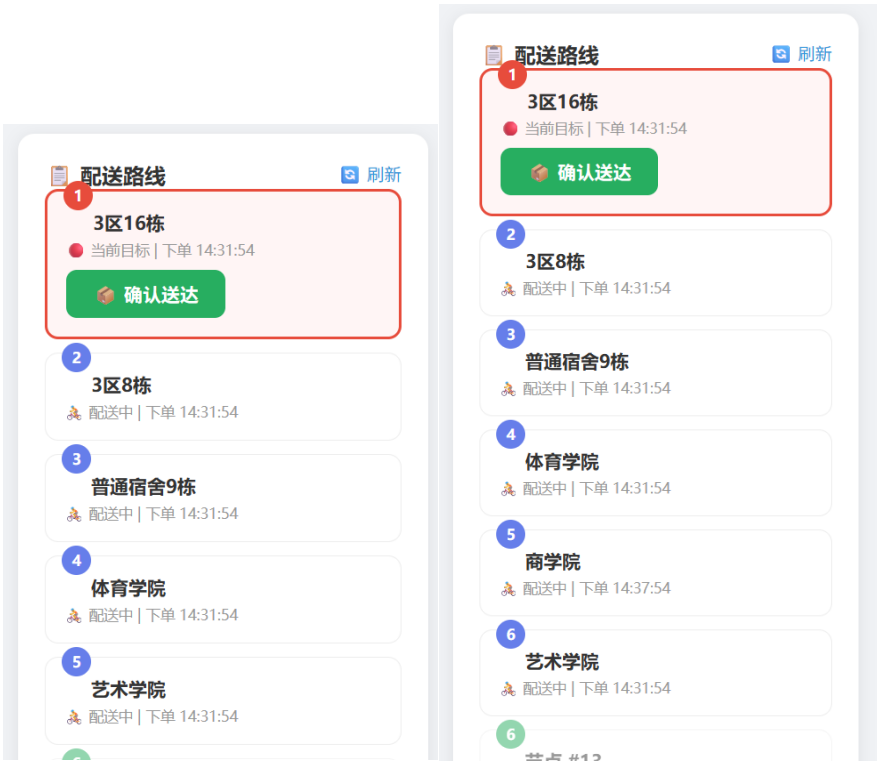
(b) 对应地图上的配送路线

图 5-11 骑手 1 分配路线

启动动态调度模式，新增订单可动态添加到骑手未到过的线路中。界面提供实时下单、随机加单（用于调试）功能，并通过 3 秒轮询显示骑手进度、冻结状态和路线统计信息。动态调度模块界面如图 5-12 所示，动态调度模块界面位于管理面板的动态调度面板中。图 5-13 演示了骑手界面配送路线动态插入了一条订单的过程（新插入了一条下单时间为 14:37:54 的送往商学院的订单），对应配送路线变化如图 5-14 所示，黄色路线和箭头表示骑手配送路线和方向，红色路线表示发生变化的部分，图 5-14(b)绿色星星表示新加入订单对应的商学院的位置。



图 5-12 动态调度（管理员端）



(a) 启动动态调度前

(b) 启动动态调度后

图 5-13 动态调度（骑手端）



(a) 启动动态调度前

(b) 启动动态调度后

图 5-14 骑手配送路线在动态调度前后的变化

5.2.4 核心实现方法

1、利用分层解耦的思想，将业务逻辑抽离到 `order_service` 服务层，视图则只负责接受请求、调用服务和返回响应。

2、双模式定位：支持两种方式都额定兴趣点的地理位置。一是直接给出 `node_index`，系统自动查找对应经纬度；二是用户给出经纬度，系统通过 `_find_nearest_node_index` 自动匹配最近图节点，再转化为标准节点坐标。

6 系统测试

6.1 注册登录模块测试

此节测试包含两部分，一部分是对外卖配送系统用户注册功能的测试，另一部分是对用户登录系统的测试。测试用例及结果如表 6.1 所示。

表 6.1 注册登录系统测试用例表

测试模块	输入/动作	输出/结果	实际情况
用户注册模块	正常注册普通用户	注册成功	一致
	正常注册骑手	注册成功	一致
	手机号已注册	注册失败	一致
	手机号格式错误	注册失败	一致
用户登录模块	普通用户正常登录	登录成功并跳转	一致
	骑手用户正常登录	登录成功并跳转	一致
	管理员正常登录	登录成功并跳转	一致
	密码错误	登录失败	一致
	手机号未注册	登录失败	一致

6.2 普通用户端功能测试

此节测试包含 4 个部分，分别是：查看配送地点、创建订单、查看订单列表、取消订单。测试用例及结果如表 6.2 所示。

表 6.2 普通用户端测试用例表

测试模块	输入/动作	输出/结果	实际情况
配送地点查看	正常获取可配送地点列表	返回列表仅含兴趣点的记录	一致
	骑手访问用户端	返回 403 Forbidden	一致

续表 6.2 普通用户端测试用例表

测试模块	输入/动作	输出/结果	实际情况
订单创建	正常下单	显示下单成功	一致
订单列表模块	查看自己的订单列表	订单列表成功显示	一致
	正常取消 pending 订单	返回“订单已取消”	一致

6.3 骑手端功能测试

此节测试包含 4 部分，分别为：查看配送任务测试、一键取餐测试、一键出发配送测试、逐单确认送达测试。测试用例及结果如表 6.3 所示。

表 6.3 骑手端测试用例表

测试模块	输入/动作	输出/结果	实际情况
配送任务模块	正常获取配送列表	配送人物列表成功显示	一致
	正常一键取餐	显示取餐成功	一致
取餐送达模块	正常一键出发	显示已经出发	一致
	正常确认单笔送达	返回送达成功，订单状态改变	一致

6.4 管理员端功能测试

此节测试包含 4 个部分，分别为兴趣点管理测试、用户管理测试、系统统计数据测试。测试用例及结果如表 6.4 所示。

表 6.4 管理员端测试用例表

测试模块	输入/动作	输出/结果	实际情况
兴趣点管理模块	点击地图某一确定位置	自动选中最近节点	一致
	正常新建兴趣点	成功新建兴趣点	一致
	新建已有的兴趣点	创建失败	一致

续表 6.4 管理员端测试用例表

测试模块	输入/动作	输出/结果	实际情况
用户管理模块	获取所有用户列表	成功获取	一致
	删除用户	成功删除	一致
统计数据模块	查看本次 pending 统计图	成功获取	一致

7 总结与展望

7.1 工作总结

本系统是为校园配送场景设计的具有路径优化功能的智能配送系统。下面总结以下本系统的主要研发工作。

首先对校园配送的需求特征进行了十分清楚、有逻辑的拆解，理顺了用户、骑手、管理员等三个角色的交互关系。在架构实现上，本系统采用了 Flask 应用工厂模式以及蓝图机制，将用户管理、骑手调度与路径优化等模块做了物理隔离的设计，这样做不仅提升了代码的复用性，也为以后功能的扩展打下了良好的基础。

由于要保证算法的实用性，本系统直接利用了 OpenStreetMap 的数据构建了校园路网模型。用 Haversine 公式实现了坐标吸附的处理。再用 Dijkstra 算法预处理节点间的最短路径。从而自然建立起来一套完备、准确的距离矩阵，为上层算法准备了准确的数据。

本系统在路径规划中采用了遗传算法与 2-opt 局部搜索相结合的混合策略。针对旅行商问题的特点，系统在遗传算法中设计顺序交叉及若干变异算子，又引入了精英保留、早停两种机制来平衡收敛速度和全局搜索的能力。最后用 2-opt 搜索算子对所得路径做平滑处理，由此很好地消除了路径交叉问题，大大提高了求解质量。

配送途中新增订单的场景，宜用基于“部分冻结”策略来设计动态调度方案。保持已经执行过的订单不变的同时，用最优插入算法进行新任务分配。又辅以定期全局微调的手段，因而能很好地兼顾毫秒级响应速度与配送路线的整体效率。

从功能测试、仿真验证两方面对系统订单管理、路径优化、动态调度各部分的稳定性做了周密分析后，我们得出了系统能缩短配送路程、提高作业效率的结论。

作者认为本系统的核心价值在于：把混合优化算法顺利地落地到了校园真实路网，且由此得出了处理动态增单需求的完整配送优化方案。

7.2 工作展望

虽然本系统对校园配送路径优化的方案做了可行性验证，但是毋庸置疑，实际应用中环境极其复杂，故今后宜从若干方面作更充分、更细致的考察。

由于目前系统是以“总配送距离最小化”作为主要优化目标，故在实际应用中显得过于单一，因此本文拟引入多目标优化方法，把配送时效性、骑手负载均衡度、用户期望满意度都合理地纳入决策模型中。

由于现有路径规划都是基于静态路网数据，因此考虑后面接入湖大教学区课间高峰时段的人流密度等实时交通信息，用机器学习方法预测路段拥堵状态，再据此动态、合理地调整路网中各边的权重。由于现有系统的逻辑是以单一食堂为中心的，故今后要向多食堂、多商户协同调度的方向扩展，因此首先要让算法能处理带容量约束的车辆路径问题，再据此研究协同配送策略。

在应用层面，本人认为系统与骑手交互的效率有待提升，可以引入语音导航或者增强现实等。同时，可以尝试添加基于强化学习的智能推荐算法，给用户最优下单时段的建议，做到需求端的错峰调节。

算法的最终性能需要在大规模实测中验证。作者希望让系统在真实校园环境下的长期运营，收集第一手反馈数据，建立完整的评价指标体系。通过对比不同参数配置下的实际增益，持续对底层算法进行调整。

参考文献

- [1] 李章恒. 校园外卖系统设计与实现[D]. 济南: 山东大学, 2022.
- [2] 赵家儒. 基于遗传算法的外卖配送路径规划[D]. 南充: 西华师范大学, 2022.
- [3] 龚艺, 冉金超, 侯明明. 基于遗传算法的多目标外卖路径规划[J]. 电子技术与软件工程, 2019(10): 157-159.
- [4] SILVA ARAÚJO K, BARBOZA F M. PSO and the traveling salesman problem: an intelligent optimization approach[J/OL]. Qeios, 2025.
- [5] 唐传茵, 章明理, 李静红, 等. 基于改进蚁群算法的外卖配送路径规划研究[J]. 南京信息工程大学学报(自然科学版), 2024, 16(2): 145-154.
- [6] 丁艳. 校园食堂外卖的系统设计与实现[J]. 数字技术与应用, 2020, 38(5): 166, 168.
- [7] 冼德锬. AES 算法在计算机数据库加密系统中的应用研究[J]. 电脑编程技巧与维护, 2025, (10): 91-93. DOI:10.16184/j.cnki.comprg.2025.10.017.
- [8] 施常月, 秦小麟, 许建秋, 等. 基于位置范围的道路网 skyline 查询[J]. 计算机科学, 2014, (9): 190-195.
- [9] DIJKSTRA E W. A note on two problems in connexion with graphs[J]. Numerische Mathematik, 1959, 1(1): 269-271.
- [10] 鲍立婷. 粒子群算法在基于 LBS 快递派送中的应用研究[D]. 东华理工大学, 2016.
- [11] 张达. 遗传算法及其发展状况研究[C]//2004 年全国理论计算机科学学术年会. 2004-10-22.
- [12] 刘飞. 遗传算法实现过程详解[J]. 福建电脑, 2010, (1): 37.
- [13] 罗继曼, 刘丰源. 基于改进蚁群算法的机器人路径规划研究[J]. 沈阳建筑大学学报(自然科学版), 2024, 40(6): 1126-1136.
- [14] 赵斌. AFL 模糊测试系统的优化方法研究[D]. 华中科技大学, 2017.
- [15] 姜昌华, 胡幼华. 一种求解旅行商问题的高效混合遗传算法[J]. 计算机工程与应用, 2004, (22): 67-71.
- [16] 韩鹏. 数据共享环境下装备制造供应链协调调度研究[D]. 大连理工大学, 2022.